

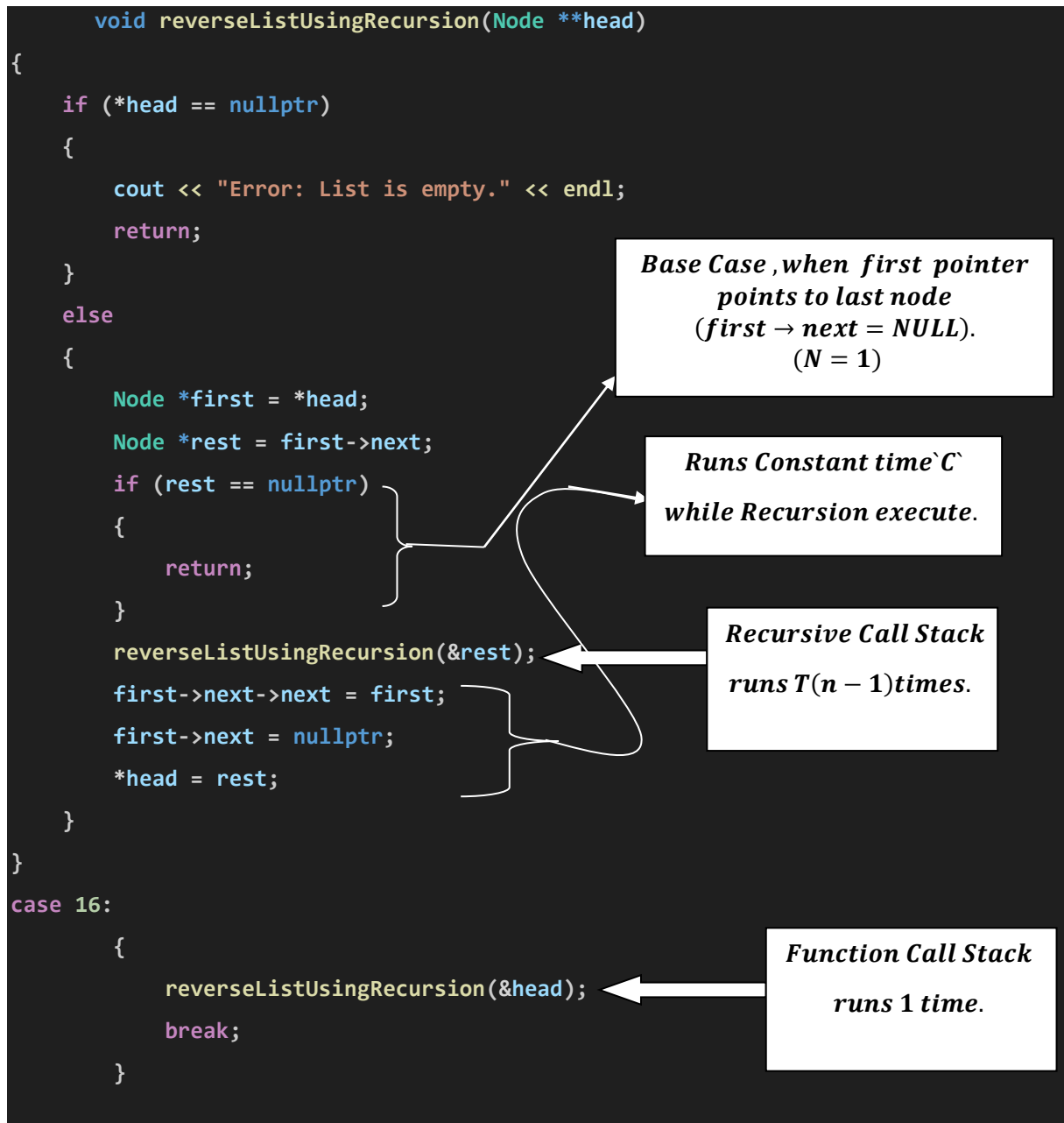
Singly Linked List – Reverse List Using Recursion and Time Complexity.

Program: –

```
void reverseListUsingRecursion(Node **head)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
    else
    {
        Node *first = *head;
        Node *rest = first->next;
        if (rest == nullptr)
        {
            return;
        }
        reverseListUsingRecursion(&rest);
        first->next->next = first;
        first->next = nullptr;
        *head = rest;
    }
}

.....
case 16:
{
    reverseListUsingRecursion(&head);
    break;
}
```

Time Complexity Analysis



In computer science, a base case is simply any condition where a recursive function returns an answer without calling itself again. Because this if statement triggers a return without making a recursive call, it completely qualifies as a base case.

However, developers often use specific terminology to separate the two base cases in your function:

1. The "Edge Case" or "Guard Clause" ($N = 0$)

```
if (*head == nullptr)
{
    cout << "Error: List is empty." << endl;
    return;
}
```

Even though this is technically a base case, programmers usually call this a Guard Clause or an Edge Case.

- *It is guarding the function from crashing if someone accidentally passes it an empty list.*
- *It handles a scenario that isn't really part of the normal reversing process. If you pass an empty list, it doesn't need to do any work; it just aborts immediately. (This is the $T(0)$ scenario).*

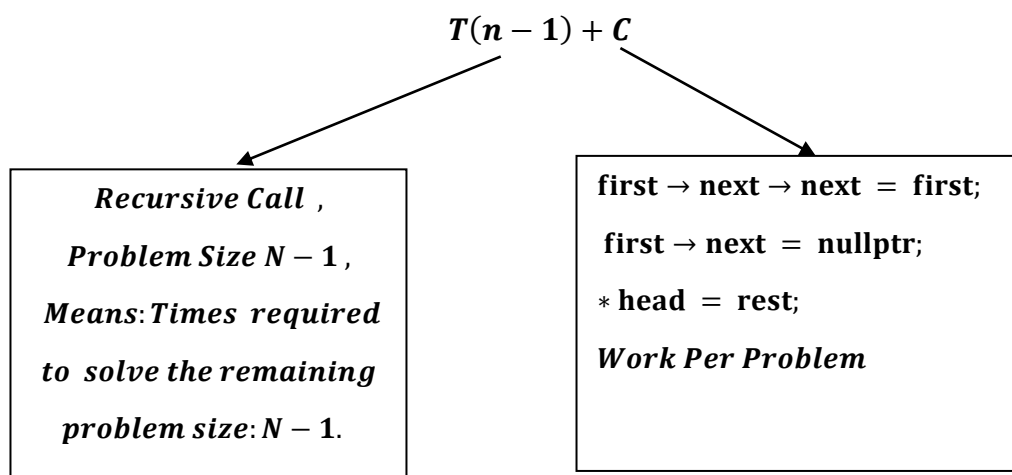
2. The "True Recursive" Base Case ($N = 1$)

```
if (rest == nullptr)
{
    return;
}
```

This is what programmers usually mean when they say The Base Case for this specific algorithm.

- *It doesn't just handle bad input; it is actively responsible for stopping the normal, healthy recursive loop once it reaches the end of a valid list. (This is the $T(1)$ scenario).*

$$T(n) = \begin{cases} 1 & , \text{for } (n = 1) \\ T(n - 1) + C & , \text{for } (n > 1) \end{cases}$$



The Work per Problem (The Chores):

```
first->next->next = first;  
first->next = nullptr;  
*head = rest;
```

While the current function frame waits for the recursion to finish and return, what physical work does the CPU do? It just shuffles three pointers. Pointer assignment is a basic, fixed operation. Therefore, the work per problem is constant time: C (or 1).

Hence: $T(n - 1) + C$

By Substitution Method

$$T(n) = T(n - 1) + C$$

$$\text{For, } T(n - 1) = T(n - 2) + C$$

Substituting $T(n - 1)$ in $T(n)$:

$$T(n) = T(n - 2) + C + C = T(n - 2) + 2C$$

$$\text{For, } T(n - 2) = T(n - 3) + C$$

Substituting $T(n - 2)$ in $T(n)$:

$$T(n) = T(n - 3) + C + 2C = T(n - 3) + 3C$$

if it goes for , level `k` , then:

$$T(n) = T(n - k) + kC$$

$$\text{As, Base Case} = T(1) \therefore n - k = 1 \Rightarrow k = n - 1.$$

Applying $k = n - 1$ in $T(n)$, we get:

$$\begin{aligned} \Rightarrow T(n) &= T(n - (n - 1)) + (n - 1)C \\ \Rightarrow T(n) &= T(1) + (n - 1)C \end{aligned}$$

$T(1)$ is base case, hence constant ' C_1 ' time or 1 unit time.

$$\Rightarrow T(n) = C_1 + (n - 1)C$$

As, C and C_1 both are constant Applying Big O, we get:

$$\Rightarrow O(N - 1)$$

$$= O(N)$$

By Recurrence Tree

<i>Level</i>	<i>No. of problems</i>	<i>Problem Size</i>	<i>Work /Cost Per Problem Size</i> <i>(Assignment of Pointers)</i>	<i>Work done = Work per Problem × No. of Problems</i>
0	1	n	C	$1 \times C = C$
1	1	$n - 1$	C	$1 \times C = C$
2	1	$n - 2$	C	$1 \times C = C$
.	.	.	.	.
.	.	.	.	.
$n - 1$ (Base Case)	1	1 (Base Case)	$T(1)$	$1 \times T(1) = T(1)$

Why level ' $n - 1$ ' is Base Case?

Ans : for , level ' k ' , Base Case = $T(1) \therefore n - k = 1$

$\Rightarrow k = n - 1$.

Total Cost = $C + C + C + \dots (N - 1)$ times + $T(1)$.

= $C(N - 1) + T(1)$

We know , $T(1)$ is 1 instead lets write $T(1) = C_1$,

representing constant.

$$= C(N - 1) + C_1$$

As, C and C₁ both are constant Applying Big O , we get:

$$O(N - 1)$$

$$= O(N)$$
